

Tutorial: Using OpenTelemetry Metrics in JavaScript

[Full-stack-observability](#)[metrics](#)[Opentelemetry](#)[Haddas Bronfman](#)*Thursday, March 16th, 2023*

🕒 6 min read

In this tutorial, we will create a Node application that demonstrates the usage of OpenTelemetry metrics. We will:

[Share](#)

- create an express server in javascript
- count the number of requests it gets per name
- send this metric to Grafana.



Whether you are familiar with OpenTelemetry and even know how to create traces in your Node app, or you never heard about OpenTelemetry and its capabilities, this tutorial will lead you step-by-step to create metrics by yourself.

Requirements

- Have a docker running on your machine
- Basic knowledge of javascript. node version $\geq 14.x.x$

Intro to OpenTelemetry

OpenTelemetry is an open-source set of tools, APIs, and SDKs that enables you to generate telemetry data about your application. This telemetry data can be logs, traces or metrics (or any combination of those).

Originally OpenTelemetry was focused on tracing and a lot of tutorials have been written about the creation of traces using OpenTelemetry API. But nowadays the metrics API has become stable, gaining more and more popularity, and could be a good alternative to other common tools (e.g. Prometheus SDK). Therefore, this tutorial will focus on the metrics capability and show you how to create metrics inside your application using OpenTelemetry.

Metrics are numerical values that represent measurements of our application performance or behavior over time. We can generate relevant measurements about our application, such as:

- Request latencies: The time it takes for requests to be processed by the application.
- Error rates: The number of errors that occur in the application, as well as their types.
- Resource usage: Such as CPU usage, memory consumption, or disk space usage.
- Throughput: The number of requests that the application is handling per second.
- Business metrics: A custom metric that is specific to the application or business, such as the number of users, revenue, etc.

What Are Metrics? What Do They Give Us?

Who Creates the Metrics?

OpenTelemetry metrics can be created in two ways:

1. Manually. As we will see in this tutorial
2. Automatically, by using OpenTelemetry instrumentations that give us metrics out of the box. There are only a few examples so far in Nodejs such as [HTTP](#) and [MySQL](#).

Using OpenTelemetry is the modern way to generate metrics, which can also interface with other OTel signals like logs and traces, which practically means you can handle all the signals using one tool.

Example Application

[Share](#)

Step 1: Create an Express Server

So like any good code example, suppose we have an express server. This server code listens to requests on this route: <http://localhost:8081/user/name>.



Run

```
$ npm init
```

to create a node application, and copy the below file to your project:

```
//server.js

const express = require('express');
const app = express();

app.get('/user/:name', (req, res) => {
  res.send("Hello " + req.params.name);
});

app.listen(8081, () => {
  console.log('Server is up and running');
});
```

Now let's use OpenTelemetry metrics API and generate some metrics.

We can measure several things in this app, according to our needs, but for simplicity, we will count the number of HTTP requests that our server gets for each different name.

Step 2: Create an OpenTelemetry Meter

Before we generate metrics, we need to create a meter object that enables us to do so. Let's see how we do it:

Copy this file to your project: (make sure to install the required libraries)

```
//meter.js

'use strict';

const { Resource } = require('@opentelemetry/resources');
const { metrics } = require('@opentelemetry/api');
const { OTLPMetricExporter } = require('@opentelemetry/exporter-metrics-otlp-grpc');
const { MeterProvider, PeriodicExportingMetricReader } = require('@opentelemetry/sdk-metrics');

const meterProvider = new MeterProvider({
  resource: new Resource({'service.name': 'my-express-app'})
});
const metricExporter = new OTLPMetricExporter();
const metricReader = new PeriodicExportingMetricReader({
  exporter: metricExporter,
  exportIntervalMillis: 60000,
});
meterProvider.addMetricReader(metricReader);
metrics.setGlobalMeterProvider(meterProvider);
```

Share



Let's explain what this code is doing:

When we come to creating a metric, there are 3 main building blocks we should be aware of:
MeterProvider → MetricsReader → MetricsExporter.

Let's elaborate on each of them:

MeterProvider is the object that holds all the metrics we create.

MetricReader is the object that reads the accumulated metrics from the provider each **exportIntervalMillis** milliseconds and sends them to the **MetricsExporter**, which in turn, sends the metrics to OpenTelemetry Collector using the gRPC protocol.

For whom is regular to Prometheus, the OpenTelemetry collector in this example is working in push and not in poll mode. That is, our application sends the metrics directly to the OpenTelemetry Collector rather than Prometheus which periodically polls the application.

The call to **setGlobalMeterProvider** sets the meter we created to be a global meter. This means that we can access metrics from anywhere in our application without having to pass them explicitly

It's important to know that when a metric is read, it is not “deleted” from the provider. The provider will keep holding it and the reader will keep reading it each 60000 milliseconds. In other words: the metrics we collected will stay for the lifetime of the provider.

Step 3: Add Metrics to the Server

Now let's modify our first server.js file, and add code that counts the number of requests:

```
//server.js

require('./meter');
const { metrics } = require('@opentelemetry/api');

const meter = metrics.getMeter('express-server');
let counter = meter.createCounter(
  'http.server.request_per_name.counter',
  {
    description: 'The number of requests per name the server got',
  }
);

const express = require('express');
const app = express();
app.get('/user/:name', (req, res) => {
  counter.add(1, {
    'route': '/user/:name',
    'name': req.params.name
  });
  res.send("Hello " + req.params.name);
});

app.listen(8081, () => {
  console.log('Server is up and running');
});
```

Share



Let's review the code we added:

The first line imports the 'meter.js' file, which actually calls the code that sets the meter provider to be the global meter. After we did that, we can call `getMeter()` and get it.

Our goal is to count the number of requests our server gets **per name**. OpenTelemetry suggests 2 types of counters: **Counter** and **UpDownCounter**. The difference between them is that the latter can also **go down**. So when should we use each of them? it depends on our needs.

If we count things like: "number of requests", "number of errors of type 5xx" or "number of accounts created" we'll use **createCounter**, but if we count the "number of active users" or "number of connections" then the **createUpDownCounter** should be used because this is a value that can be incremented or decremented.

The next part is the one that actually counts what we need:

```
counter.add(1, {
  'route': '/user/:name',
  'name': req.params.name
});
```

The first parameter here is the number that we want to add to the counter – which can be negative if you use **UpDownCounter** – and the second parameter is a dictionary that characterizes the metric.

Each unique dictionary will create a different timeseries we will later see in Grafana.

Run the Server

run your server:

```
$ node server.js
```

Go to <http://localhost:8081/user/yourName> so the server will get a request. To make it interesting, send several requests with different names. behind the scene, OpenTelemetry created a timeseries for each name. We will soon see it visually in Grafana 🖼️😊

OK. We Created Metrics, but Where Can We See Them?

By default, the **metricsExporter** sends the metrics to an **OpenTelemetry Collector** running locally on your machine. The collector is the part that collects all the telemetry data we created. It can collect data from several applications and it can also collect any kind of telemetry data – metrics, logs or traces.

The collector can export the data to a wide list of available backends. A list of them can be found [here](#) (select a specific exporter, then look at the ‘Supported pipeline types’ field to see if this exporter supports metrics and/or traces/logs). In this example, we’ll use Prometheus since it is widely used, and Grafana will query the metrics from there.

Share



To ease this stage I created a docker-compose.yaml for you that set up OpenTelemetry Collector, Prometheus, and Grafana.

Please copy [this](#) docker folder to your local machine. it can be inside your project or anywhere else on your computer. From the location of docker-compose.yaml file, run:

```
$ docker compose up
```

Give it a minute to run. Run `$ docker ps` and verify you see OTel collector, Prometheus, and Grafana.

See Metrics on Grafana:

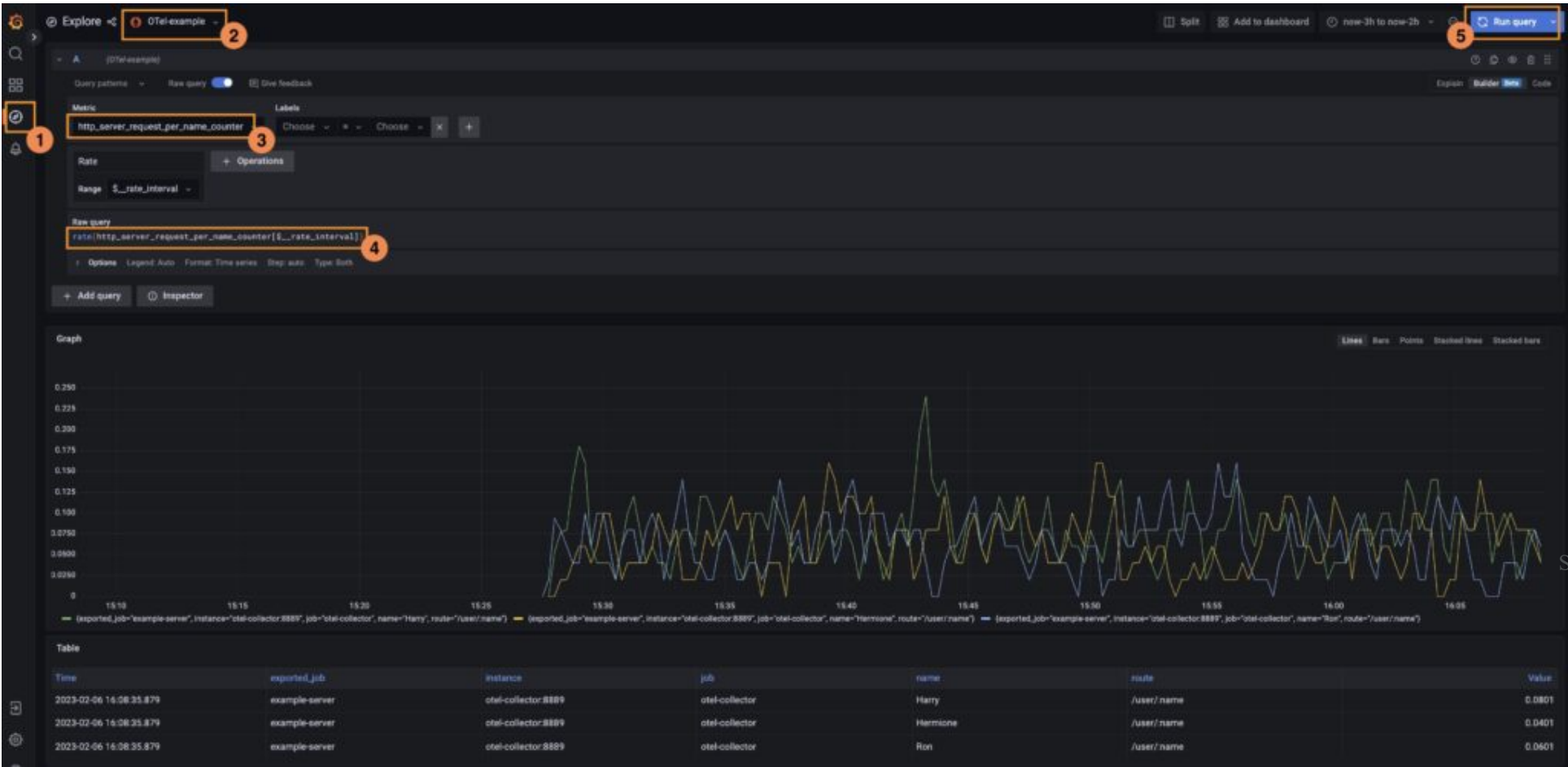
For those who know Grafana, you probably know where to find your metrics already.

For those who don’t, here is the explanation:

Grafana is available here: <http://localhost:3000/>

To see your metrics follow the steps:

1. Press ‘Explore’
2. Choose the data source ‘OTel example’
3. Choose the metric name you created in the app
4. Add rate() to get better visualization
5. Run the query.



And yes! we have the metric we created!

Summary

We saw how OpenTelemetry metrics API enables us to measure relevant information about our application. We can create one ‘meter.js’ file in our application to set the global meter, then use that meter from anywhere in our app where we want to count or measure something. We also saw how the metrics can be easily seen in Grafana.

Keep in mind, that many trivial measurements come automatically when using OpenTelemetry instrumentations (which we didn’t cover in this tutorial). So if you look for a specific measurement, check first if it already exists in the library you use.

It is important to say, that OpenTelemetry suggests more measurements besides counting, such as histograms or gauges. for further reading see [here](#).